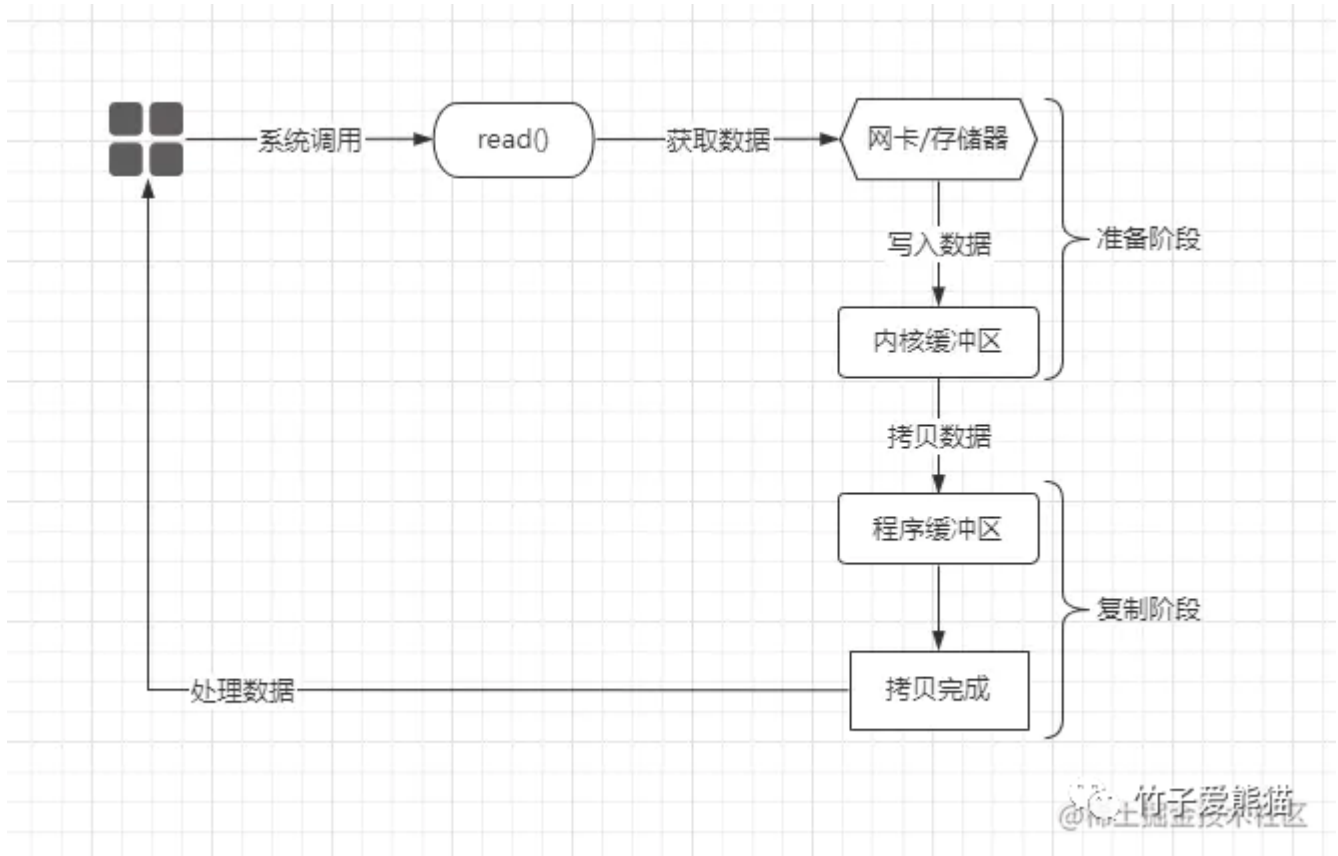


## 8.14

### (同步)阻塞模型（一直等）

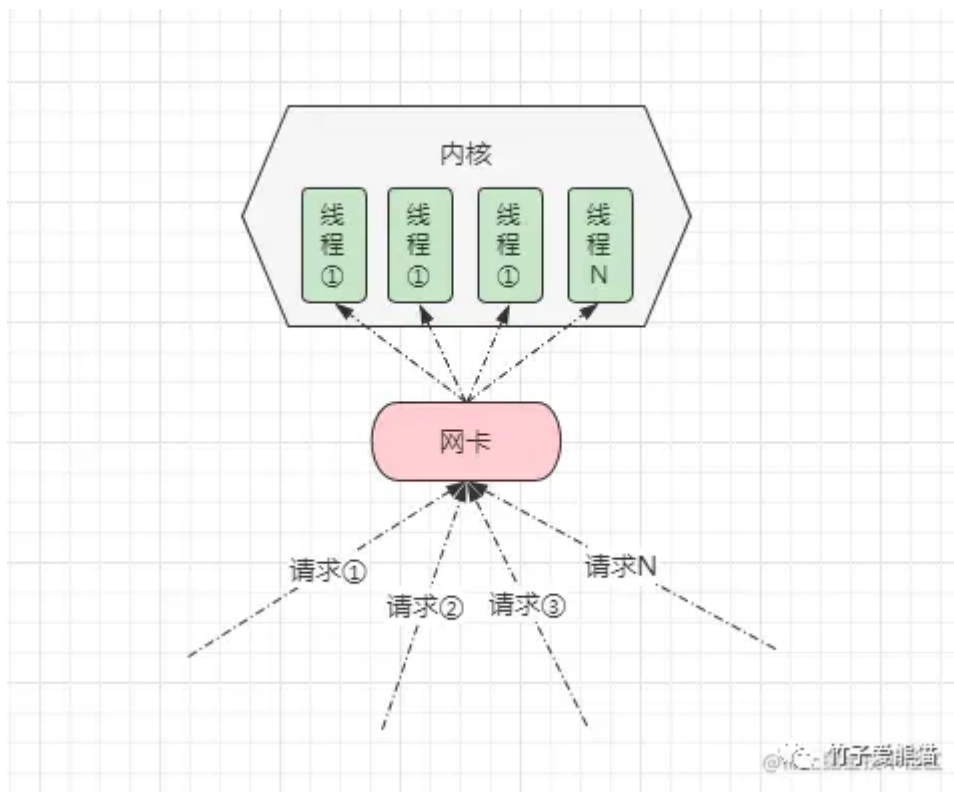
也就是当调用 内核的 `read()` 函数后，内核在执行数据准备、复制阶段的 IO 操作时，应用线程都是阻塞的；



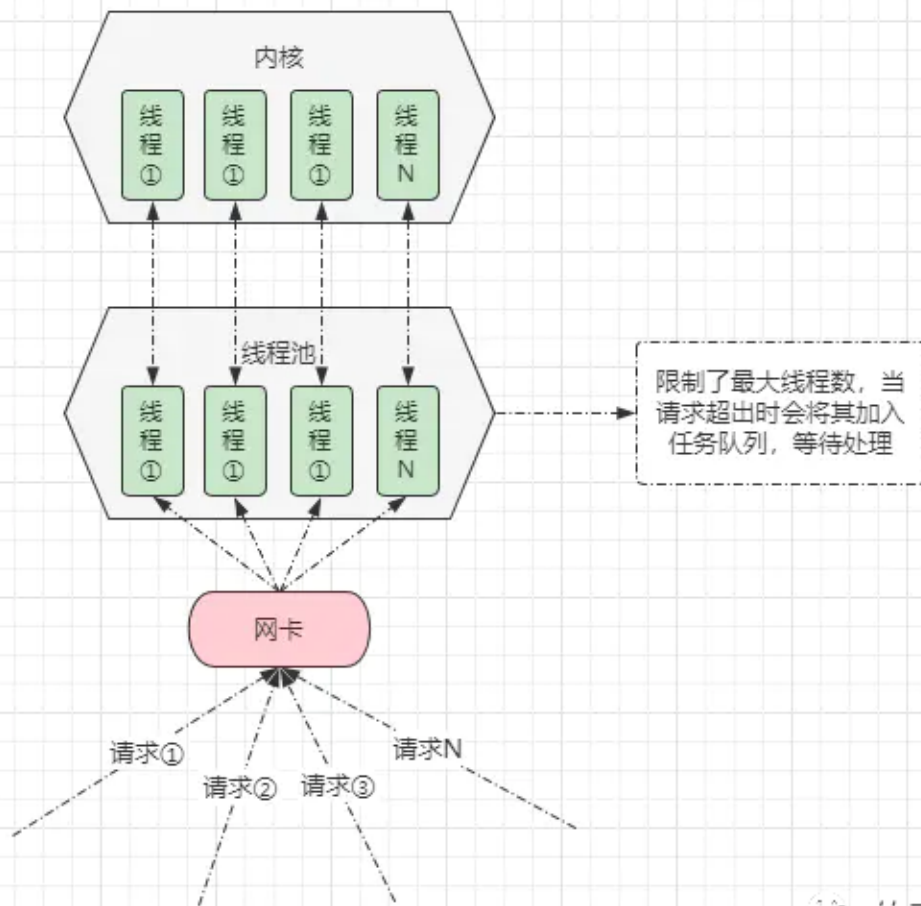
当程序中需要进行 IO 操作时，会先调用内核提供的 `read()` 函数，但在之前分析过 IO 的工作原理，IO 会经过“设备→内核缓冲区→程序缓冲区”这个过程，该过程必然是耗时的，在同步阻塞模型中，程序中的线程发起 IO 调用后，会一直挂起等待，直至数据成功拷贝至程序缓冲区才会继续往下执行。

在一般的 同步阻塞模型中，为了支持并发请求，通常会采用“请求：线程”1:1 的模型，那此时会带来很大的弊端：

- 并发过高时会导致创建大量线程，而线程资源是有限的，超出后会导致系统崩溃。
- 并发过高时，就算创建的线程数未达系统瓶颈，但由于线程数过多也会造成频繁的上下文切换。



有一个优化方案就是采用线程池；

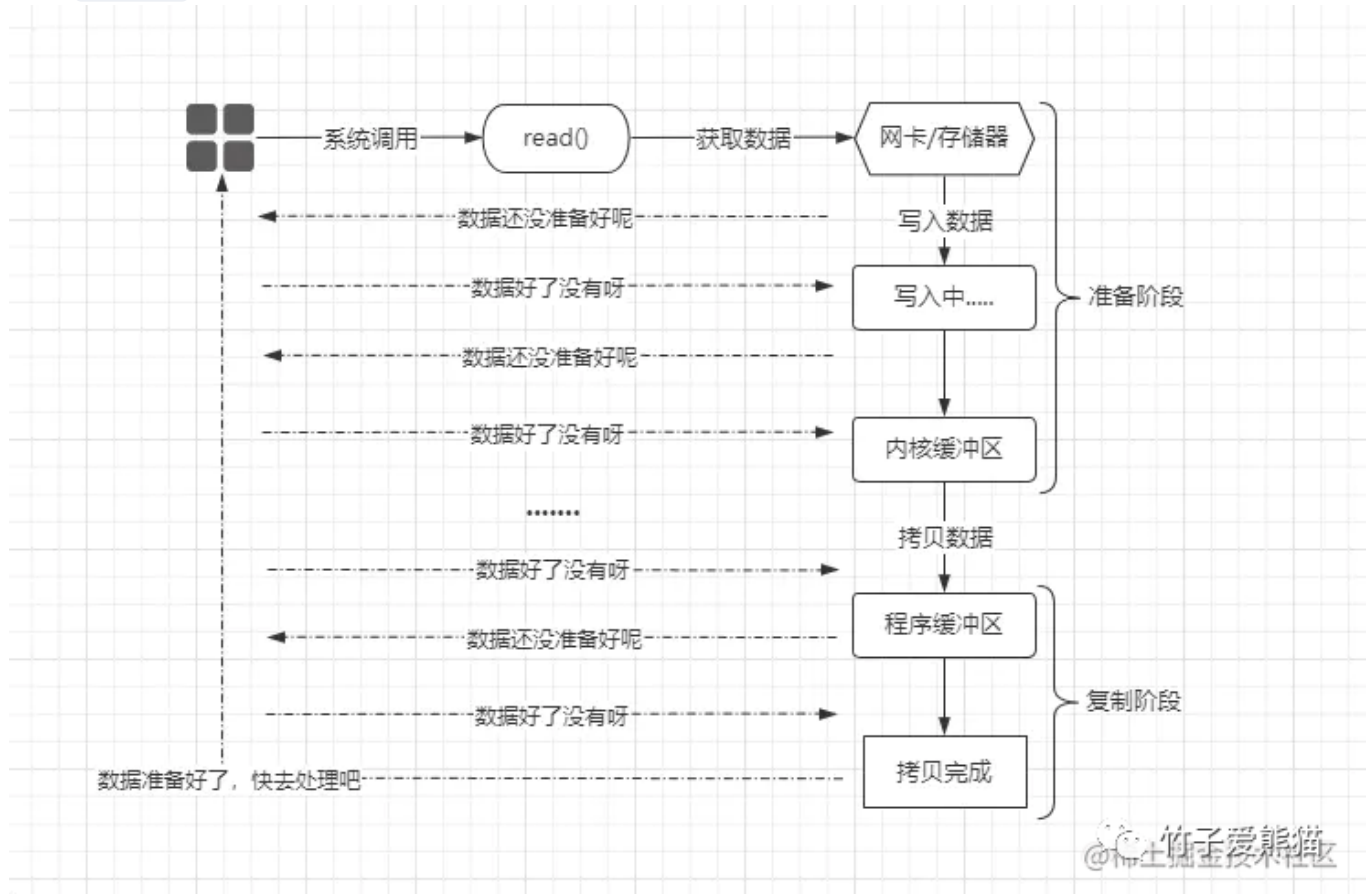


@竹子爱熊猫

## (同步)非阻塞模型 (不等, 不断轮询)

`setoption(socket);`将套接字设为非阻塞模式

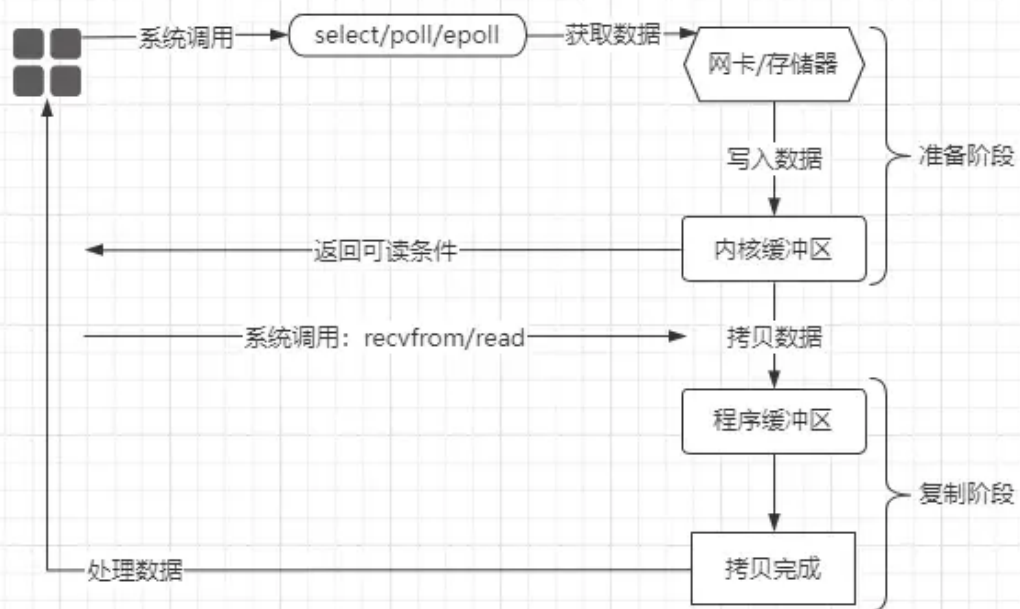
调用 `read()` 函数的线程并不会阻塞，而是可以正常运行



当应用程序中发起 `io` 调用后，内核并不阻塞当前线程，而是立马返回一个“数据未就绪”的信息给应用程序，而应用程序这边则一直反复轮询去问内核：数据有没有准备好？直到最终数据准备好了之后，内核返回“数据已就绪”状态，紧接着再由进程去处理数据.....

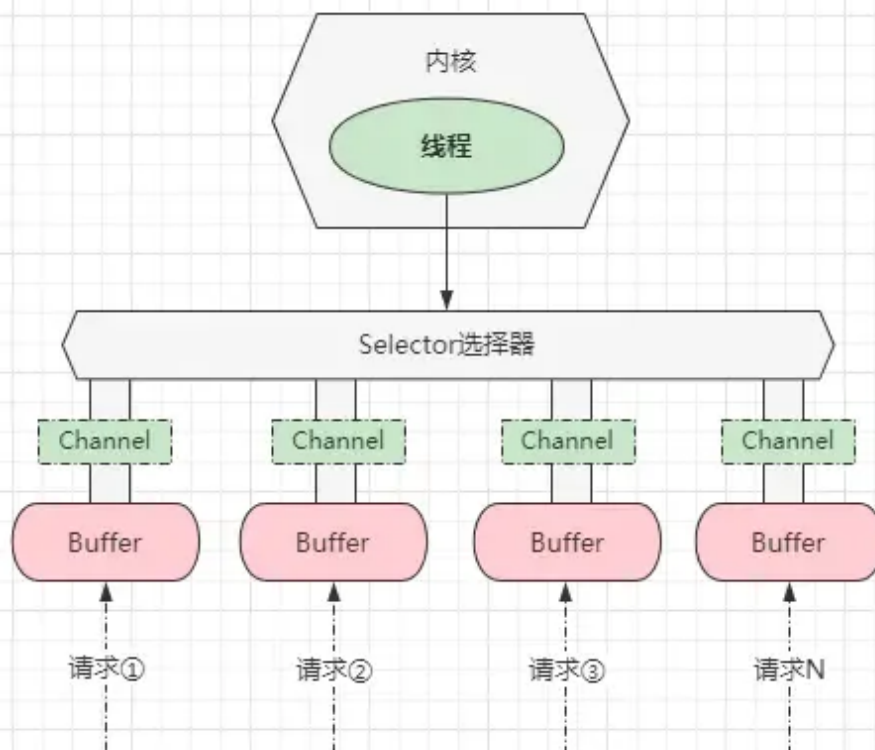
## (同步)多路I/O复用（不轮询，等系统通知）

指使用一个线程处理多个I/O操作(channel)（复用一个线程）（eg、多个套接字），避免高并发场景下，为每个socket分配一个线程，造成线程浪费



竹子爱熊猫  
@稀土掘金技术社区

当出现一个 `IO` 操作时，会通过调用内核提供的多路复用函数，将当前连接注册到监听器上，当监听器发现该连接的数据准备就绪后，会返回一个可读条件给用户进程，用户自己拷贝数据到用户缓冲区（这里实际是读取 `Socket` 缓冲区中的数据）。



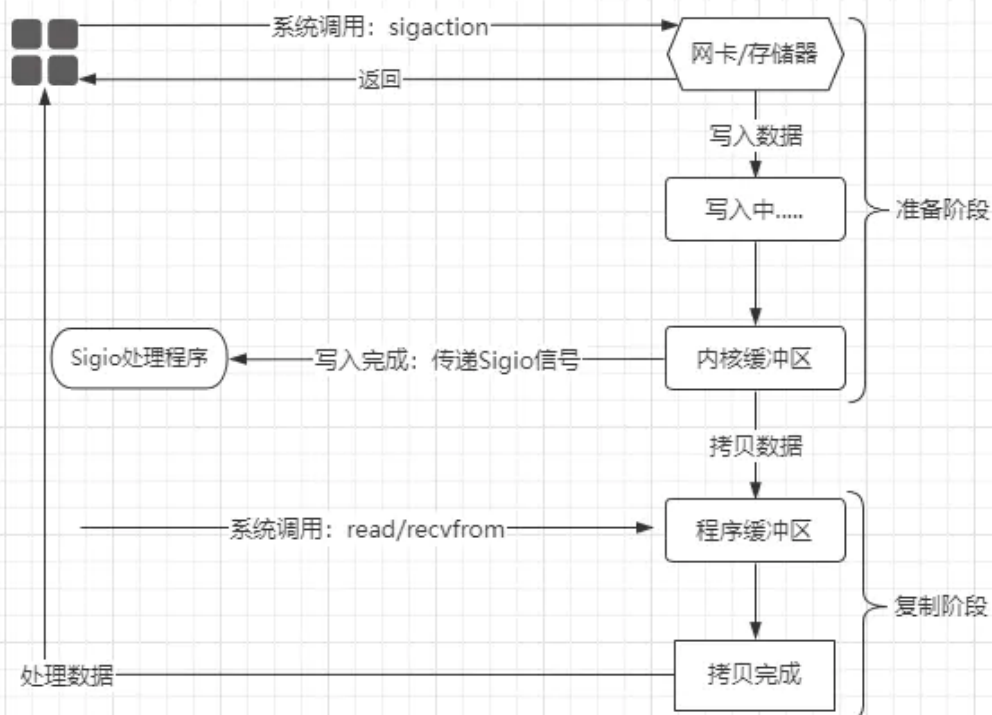
@竹子爱熊猫

在多路复用模型中，内核仅有一条线程负责处理所有连接，所有网络请求/连接（`Socket`）都会利用通道 `Channel` 注册到选择器上，然后监听器负责监听所有的连接

即把“等待 I/O 就绪”这件事集中到一个地方（`select/poll/epoll`），用极少量线程就能同时管理海量连接。

`poll`、`select`、`epoll`通过设置`timeout`超时时间进行轮询

## (异步+同步)信号驱动



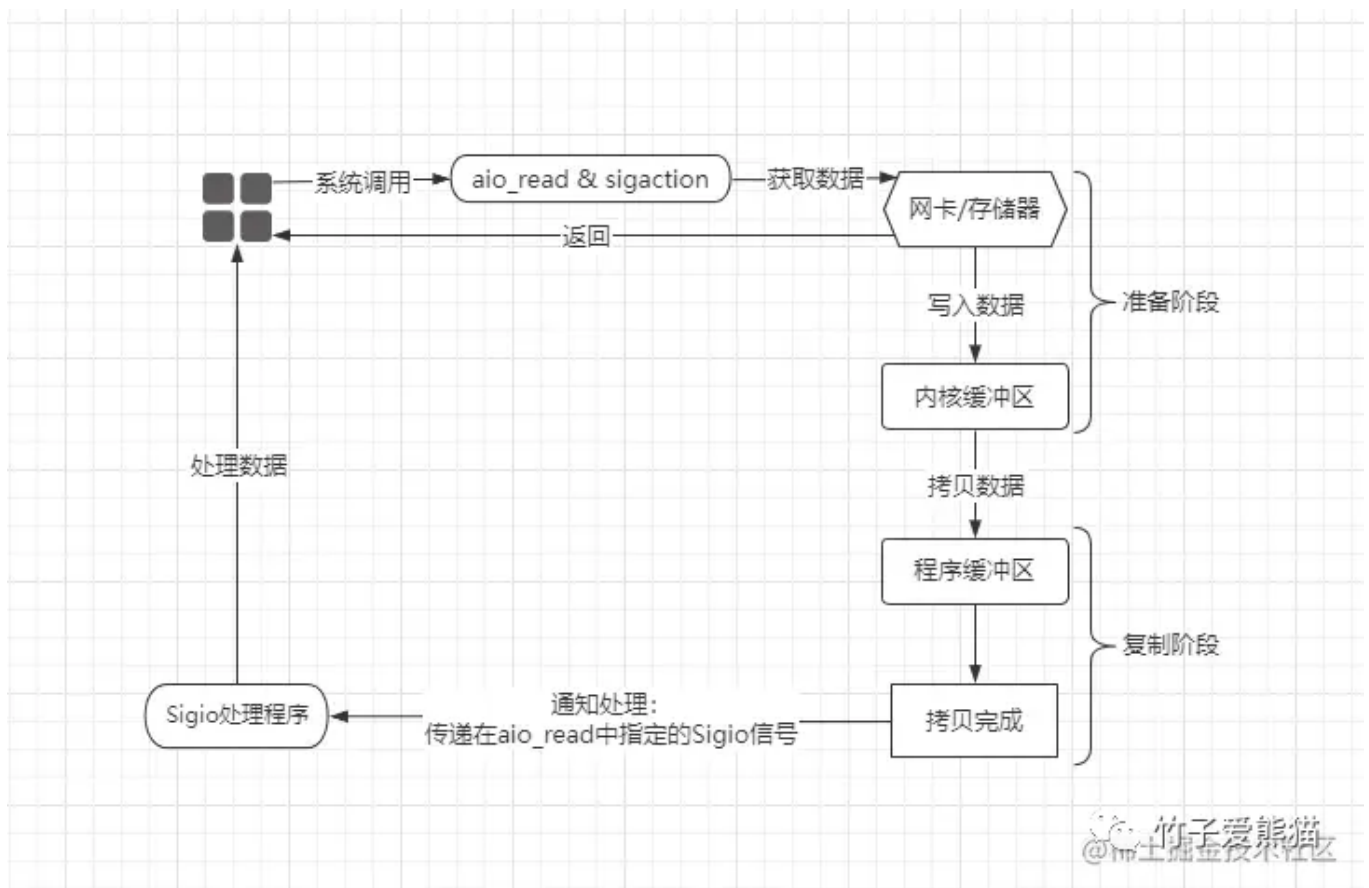
@Tiger 王爱军社区

在该模型中，首先用户进程中会创建一个 `Sigio` 信号处理程序，然后会系统调用 `sigaction` 信号处理函数，紧接着内核会直接让用户进程中的线程返回，用户进程可在这期间干别的工作，当内核中的数据准备好之后，内核会生成一个 `Sigio` 信号，通知对应的用户进程数据已准备就绪，然后由用户进程在触发一个 `recvfrom` 的系统调用，从内核中将数据拷贝出来进行处理。

信号驱动模型相较于之前的模型而言，从一定意义上实现了异步，也就是数据的准备阶段是异步非阻塞执行的，但数据的复制阶段却依旧是同步阻塞执行的。

## 异步非阻塞I/O

异步非阻塞模型，该模型是真正意义上的异步非阻塞式 `IO`，代表数据准备与复制阶段都是异步非阻塞的



同样会基于信号驱动实现，在刚开始会先调用 `aio_read`、`sigaction` 函数，然后用户进程中会创建一个信号处理程序，同时用户进程可立马返回执行其他操作，**在内核将数据写入到内核、且从内核拷贝到用户缓冲区后**，内核会通知对应的用户进程对数据进行处理。

真正意义上的实现了异步非阻塞，从始至终用户进程只需要发起一次系统调用，后续的所有 IO 操作由内核完成，最后在数据拷贝至程序缓冲区后，通知用户进程处理即可。

典型的异步非阻塞模型：Node.js windows 的 IOPC

## Reactor VS Proactor

**同步就绪通知:** 内核告诉你可以读了，**你来读**（需要用户自己将数据拷贝到用户缓冲区）

**多路IO复用** 是基于 `Reactor` 模式构建的，`Reactor` 负责事件的注册、监听、派发等工作，也就是对应着 `Selector` 选择器，它是 `NIO` 的核心。

```
Selector.select() → channel.read(buf) //非阻塞
```

支持的操作系统：select/poll/epoll(Linux/BSD); kqueue; Java NIO Select

**异步完成通知:** 内核已经帮你读完，**直接用**（内核将数据已经拷贝到用户缓冲区，用户只用处理业务逻辑）



而 异步非阻塞IO 则是基于 Proactor 模式构建的，Proactor 负责异步 IO 的回调工作派发，在 AIO 技术中，AsynchronousChannelGroup 则担任着 Proactor 的角色。

```
AsynchronousSocketChannel.read(buf, null, handler)// → 回调里直接用 buf
```

支持的操作系统：Windows IOCP（最成熟）；Linux AIO（io\_uring 正在成熟）；Java NIO2 AsynchronousChannel + CompletionHandler